
EAST WEST UNIVERSITY
Department of Computer Science and Engineering
CSE488: Big Data Analytics | Summer 2026

TERM PROJECT DESCRIPTION

A RAG-Based Expert System for Mobile/Laptop Recommendation

1. Overview

Students will design and implement a web-based conversational expert system that recommends mobile phones and laptops to end users based on stated requirements such as budget, intended use-case, and desired specifications. The system must be grounded via Retrieval-Augmented Generation (RAG) over a dataset that each team collects and curates independently. Apache Spark must be used for large-scale data processing, and FAISS must be used for vector indexing and approximate nearest neighbor (ANN) retrieval.

2. Learning Outcomes

On successful completion of this project, students will be able to:

- Apply distributed data processing techniques using Apache Spark to a real, self-collected, unstructured dataset. [CO2]
- Apply similarity-search and ANN techniques (MinHash/LSH, FAISS index families) to a production-style retrieval problem. [CO3]
- Design, implement, and evaluate a Retrieval-Augmented Generation pipeline that combines structured filtering, dense retrieval, and an LLM generator. [CO3]
- Build, integrate, and deploy a complete end-to-end web application. [CO4]

3. Project Scope and Requirements

3.1 Dataset (Publicly available dataset + Student-Owned)

- [kaggle.com/datasets/muhammetvarl/laptop-price](https://www.kaggle.com/datasets/muhammetvarl/laptop-price)
- Minimum 1,500–2,000 device entries (phones and laptops combined).
- Each entry must include structured fields (price, CPU, RAM, storage, GPU, display, battery, category tags) and unstructured text (expert/user reviews or spec descriptions).
- Data source(s) and collection methodology must be documented (scraping, public datasets, manual compilation, or a mix).

3.2 Apache Spark Processing Pipeline (Mandatory)

- Cleaning and normalization of raw collected data using the Spark DataFrame API.
- Near-duplicate detection using MinHash + LSH (Spark MLlib) applied to review or spec text.
- Text chunking implemented via Spark UDFs / mapPartitions.
- Distributed embedding generation using a pandas_udf that batches calls to a sentence-transformer model.
- Output: embeddings and associated metadata written to Parquet.

3.3 Vector Database and ANN Retrieval (Mandatory)

- Build a FAISS index over the Spark-generated embeddings.
- Implement and compare at least two FAISS index types (e.g., IndexFlatL2 baseline vs. IVF vs. HNSW vs. IVF+PQ) on recall@k and query latency.
- Provide a written justification for the final index choice given dataset size and latency requirements.

3.4 RAG Chatbot Backend

- A FastAPI service implementing: query → embed → FAISS retrieve → (optional metadata filter) → LLM generates a grounded recommendation with justification.
- Retrieved context must be displayed alongside the generated answer; unsupported/hallucinated answers are not acceptable.

3.5 Web Frontend

- A chat-style interface (Streamlit is acceptable; Flask or React is preferred for stronger teams).
- The interface should present recommended device(s) with specifications and reasoning, not free text alone.

3.6 Evaluation

- A hand-labeled query set of at least 30 queries with expected relevant devices.
- Report precision@k and recall@k for retrieval quality, plus a qualitative assessment of generated answer quality.

4. Stretch Goals (Bonus Marks)

- Hybrid search combining metadata filtering (price/category) with vector similarity search.
- FP-Growth analysis on spec co-occurrence to power a "commonly paired specifications" insight feature.
- A semantic caching layer for similar past queries, benchmarked for latency and hit-rate improvement.

5. Milestones and Grade Weighting

Milestone	Deliverable	Weight
M1	Dataset + schema design + data collection report	15%
M2	Spark pipeline: cleaning, deduplication, chunking, embedding	15%
M3	FAISS index build + comparative evaluation report	15%
M4	RAG backend + retrieval evaluation (precision/recall@k)	15%
M5	Web application + final demo + project report	40%

6. Team Composition

Teams of 3–4 students are recommended given the workload involved in dataset collection and curation. Each team member's individual contribution must be documented in the final report.

Academic Integrity Note

All code, datasets, and reports must represent the original work of the team. Use of external libraries and pretrained models (sentence-transformers, FAISS, LLM APIs) is expected and encouraged; however, the pipeline design, integration, and evaluation must be the team's own work. Any external code reused must be cited in the final report.